

PROGRAM - 1

PART - A

AIM – Write a program to demonstrate the use of conditional statements and loops.

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
using namespace std;
int main() {
    int n, sum = 0;
    cout << "Enter a positive integer: ";
    cin >> n;
    if (n <= 0) {
        cout << "Please enter a positive integer." << endl;
        return 1;
    }
    for (int i = 1; i <= n; i++) {
        if (i % 2 == 0) {
            sum += i;
        }
    }
    cout << "The sum of all even numbers between 1 and " << n << " is: " << sum <<
endl;
    return 0;
}
```

Output –

```
Enter a positive integer: 25
The sum of all even numbers between 1 and 25 is: 156

=== Code Execution Successful ===|
```

PART - B

AIM - Write a program to demonstrate the use of pointers.

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
using namespace std;
void swap(int* x, int* y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main() {
    int a, b;
    cout << "Enter first number: ";
    cin >> a;
    cout << "Enter second number: ";
    cin >> b;
    swap(&a, &b);
    cout << "First Number: " << a << endl;
    cout << "Second Number: " << b << endl;
    return 0;
}
```

Output –

```
Enter first number: 3
Enter second number: 44
First Number: 44
Second Number: 3
```

PART - C

AIM – Write a program for sparse matrix using array.-

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
using namespace std;
int main() {
    int rows, cols;
    cout << "Enter the number of rows: ";
    cin >> rows;
    cout << "Enter the number of columns: ";
    cin >> cols;
    int matrix[rows][cols];
    cout << "Enter the elements of the matrix:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cin >> matrix[i][j];
        }
    }
```

```

}
int size = 0;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] != 0) {
            size++;
        }
    }
}
int sparseMatrix[size][3];
int k = 0;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] != 0) {
            sparseMatrix[k][0] = i;
            sparseMatrix[k][1] = j;
            sparseMatrix[k][2] = matrix[i][j];
            k++;
        }
    }
}
cout << "Sparse Matrix representation:" << endl;
cout << "Row Column Value" << endl;
for (int i = 0; i < size; i++) {
    cout << sparseMatrix[i][0] << " " << sparseMatrix[i][1] << " " << sparseMatrix[i][2]
    << endl;
}
return 0;
}

```

Output –

```

Enter the number of rows: 3
Enter the number of columns: 3
Enter the elements of the matrix:
34 32 45 23 54 56 34 34 54
Sparse Matrix representation:
Row Column Value
0      0      34
0      1      32
0      2      45
1      0      23
1      1      54
1      2      56
2      0      34
2      1      34
2      2      54

```

PROGRAM - 2

AIM – Write a program to Create a linked list with nodes having information about student and perform – a) Insert a new node at a specified position

b) Delete a node with roll number specified

c) Reversal of that Linked list

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
struct Student {
```

```

    int rollNumber;
    string name;
    Student* next;
    Student(int r, string n) : rollNumber(r), name(n), next(nullptr) {}
};

class LinkedList {
    Student* head;

public:
    LinkedList() : head(nullptr) {}

    void insertAtPosition(int position, int rollNumber, string name) {
        if (position < 0) return;
        Student* newNode = new Student(rollNumber, name);
        if (position == 0) {
            newNode->next = head;
            head = newNode;
        } else {
            Student* temp = head;
            for (int i = 0; i < position - 1 && temp; i++) temp = temp->next;
            if (!temp) {
                delete newNode;
                return;
            }
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }

    void deleteByRollNumber(int rollNumber) {
        Student* temp = head, *prev = nullptr;
        while (temp && temp->rollNumber != rollNumber) {
            prev = temp;
            temp = temp->next;
        }
        if (!temp) return;
        if (prev) prev->next = temp->next;
        else head = temp->next;
        delete temp;
    }

    void reverse() {
        Student* prev = nullptr, *current = head;

```

```

        while (current) {
            Student* next = current->next;
            current->next = prev;
            prev = current;
            current = next;
        }
        head = prev;
    }

    void display() const {
        for (Student* temp = head; temp; temp = temp->next) {
            cout << "Roll Number: " << temp->rollNumber << ", Name: " << temp->name <<
endl;
        }
    }

    ~LinkedList() {
        while (head) {
            Student* temp = head;
            head = head->next;
            delete temp;
        }
    }
};

int main() {
    LinkedList list;
    list.insertAtPosition(0, 101, "Kunal");
    list.insertAtPosition(1, 102, "Mayank");
    list.insertAtPosition(2, 103, "Ishaan");

    cout << "Original list:" << endl;
    list.display();

    list.insertAtPosition(1, 104, "Happy");
    cout << "\nAfter inserting Happy at position 1:" << endl;
    list.display();

    list.deleteByRollNumber(102);
    cout << "\nAfter deleting student with roll number 102:" << endl;
    list.display();

    list.reverse();
    cout << "\nAfter reversing the list:" << endl;

```

```
list.display();  
  
return 0;  
}
```

Output –

```
Original list:  
Roll Number: 101, Name: Kunal  
Roll Number: 102, Name: Mayank  
Roll Number: 103, Name: Ishaan  
  
After inserting Happy at position 1:  
Roll Number: 101, Name: Kunal  
Roll Number: 104, Name: Happy  
Roll Number: 102, Name: Mayank  
Roll Number: 103, Name: Ishaan  
  
After deleting student with roll number 102:  
Roll Number: 101, Name: Kunal  
Roll Number: 104, Name: Happy  
Roll Number: 103, Name: Ishaan  
  
After reversing the list:  
Roll Number: 103, Name: Ishaan  
Roll Number: 104, Name: Happy  
Roll Number: 101, Name: Kunal
```


PROGRAM - 3

AIM – Write a program to Create a doubly linked list with nodes having information about employee and perform deletion at end and insertion at front.

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
#include <string>
using namespace std;
```

```
struct Node {
    int id;
    string name;
    Node* prev = nullptr;
    Node* next = nullptr;
};
```

```
Node* createNode(int id, string name) {
    return new Node{id, name};
}
```

```

void insertAtFront(Node*& head, int id, string name) {
    Node* newNode = createNode(id, name);
    newNode->next = head;
    if (head) {
        head->prev = newNode;
    }
    head = newNode;
}

void deleteAtEnd(Node*& head) {
    if (!head) return;
    Node* temp = head;
    while (temp->next) {
        temp = temp->next;
    }
    if (temp->prev) {
        temp->prev->next = nullptr;
    } else {
        head = nullptr;
    }
    delete temp;
}

void displayList(Node* head) {
    while (head) {
        cout << "Employee ID: " << head->id << ", Name: " << head->name << endl;
        head = head->next;
    }
    cout << endl;
}

int main() {
    Node* head = nullptr;

    insertAtFront(head, 101, "Kunal");
    insertAtFront(head, 102, "Mayank");
    insertAtFront(head, 103, "Ishaan");
    insertAtFront(head, 104, "Happy");

    cout << "Original List:" << endl;
    displayList(head);

    insertAtFront(head, 105, "Amit");
    cout << "List after inserting 'Amit' at the front:" << endl;
}

```

```
displayList(head);
deleteAtEnd(head);

cout << "List after deleting the node at the end:" << endl;
displayList(head);
return 0;
}
```

Output –

```
Original List:
Employee ID: 104, Name: Happy|
Employee ID: 103, Name: Ishaan
Employee ID: 102, Name: Mayank
Employee ID: 101, Name: Kunal

List after inserting 'Amit' at the front:
Employee ID: 105, Name: Amit
Employee ID: 104, Name: Happy
Employee ID: 103, Name: Ishaan
Employee ID: 102, Name: Mayank
Employee ID: 101, Name: Kunal

List after deleting the node at the end:
Employee ID: 105, Name: Amit
Employee ID: 104, Name: Happy
Employee ID: 103, Name: Ishaan
Employee ID: 102, Name: Mayank
```

PROGRAM – 4

AIM – Write a program to Create a circular linked list with nodes having information about collage and perform deletion at end and insertion at front.

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
#include <string>
using namespace std;
```

```
struct Node {
    int id;
    string name;
    Node* next;
};
Node* createNode(int id, string name) {
```

```

Node* newNode = new Node();
newNode->id = id;
newNode->name = name;
newNode->next = nullptr;
return newNode;
}
void insertAtFront(Node*& head, int id, string name) {
    Node* newNode = createNode(id, name);
    if (head == nullptr) {
        head = newNode;
        newNode->next = head;
    } else {
        Node* temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        newNode->next = head;
        temp->next = newNode;
        head = newNode;
    }
}
void deleteAtEnd(Node*& head) {
    if (head == nullptr) {
        cout << "List is empty, nothing to delete." << endl;
        return;
    }
    if (head->next == head) {
        delete head;
        head = nullptr;
        return;
    }
    Node* temp = head;
    Node* prev = nullptr;
    while (temp->next != head) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = head;
    delete temp;
}
void displayList(Node* head) {
    if (head == nullptr) {
        cout << "List is empty." << endl;
        return;
    }

```

```

    }
    Node* temp = head;
    do {
        cout << "College ID: " << temp->id << ", Name: " << temp->name << endl;
        temp = temp->next;
    } while (temp != head);
    cout << endl;
}

int main() {
    Node* head = nullptr;
    insertAtFront(head, 201, "MIT");
    insertAtFront(head, 202, "Stanford");
    insertAtFront(head, 203, "Harvard");
    insertAtFront(head, 204, "Caltech");

    cout << "Original Circular List:" << endl;
    displayList(head);

    insertAtFront(head, 205, "Oxford");
    cout << "Circular List after inserting 'Oxford' at the front:" << endl;
    displayList(head);
    deleteAtEnd(head);
    cout << "Circular List after deleting the node at the end:" << endl;
    displayList(head);
    return 0;
}

```

Output –

```

Original Circular List:
College ID: 204, Name: Caltech
College ID: 203, Name: Harvard
College ID: 202, Name: Stanford
College ID: 201, Name: MIT

Circular List after inserting 'Oxford' at the front:
College ID: 205, Name: Oxford
College ID: 204, Name: Caltech
College ID: 203, Name: Harvard
College ID: 202, Name: Stanford
College ID: 201, Name: MIT

Circular List after deleting the node at the end:
College ID: 205, Name: Oxford
College ID: 204, Name: Caltech
College ID: 203, Name: Harvard
College ID: 202, Name: Stanford

```

PROGRAM - 5

AIM – Write a program to implement two stacks in a using single array.

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

class TwoStacks {
private:
    int* arr;
    int size;
    int top1, top2;
public:
    TwoStacks(int n) {
        size = n;
        arr = new int[n];
        top1 = -1;
        top2 = size;
    }
    void push1(int x) {
        if (top1 < top2 - 1) {
            arr[++top1] = x;
        } else {
            cout << "Stack Overflow\n";
        }
    }
};
```

```

    }
}
void push2(int x) {
    if (top1 < top2 - 1) {
        arr[--top2] = x;
    } else {
        cout << "Stack Overflow\n";
    }
}
int pop1() {
    if (top1 >= 0) {
        int x = arr[top1--];
        return x;
    } else {
        cout << "Stack Underflow\n";
        return -1;
    }
}
int pop2() {
    if (top2 < size) {
        int x = arr[top2++];
        return x;
    } else {
        cout << "Stack Underflow\n";
        return -1;
    }
}
~TwoStacks() {
    delete[] arr;
}
};
int main() {
    TwoStacks ts(10);
    ts.push1(5);
    ts.push2(10);
    ts.push1(15);
    ts.push2(20);
    cout << "Popped from Stack 1: " << ts.pop1() << endl;
    cout << "Popped from Stack 2: " << ts.pop2() << endl;
    return 0;
}

```

Output –

```

Popped from Stack 1: 15
Popped from Stack 2: 20

```

```

=== Code Execution Successful ===

```


PROGRAM - 6

AIM – Write a program to Create a stack and perform pop, push, peek and traverse operation on stacks using linked list

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* next;
```

```
};
```

```
class Stack {
private:
    Node* top;
```

```

public:
    Stack() : top(nullptr) {}
    ~Stack() {
        while (!isEmpty()) {
            pop();
        }
    }
    void push(int value) {
        Node* newNode = new Node();
        newNode->data = value;
        newNode->next = top;
        top = newNode;
        cout << value << " pushed onto the stack." << endl;
    }
    void pop() {
        if (isEmpty()) {
            cout << "Stack is empty, nothing to pop." << endl;
            return;
        }
        Node* temp = top;
        top = top->next;
        cout << temp->data << " popped from the stack." << endl;
        delete temp;
    }
    void peek() const {
        if (isEmpty()) {
            cout << "Stack is empty, nothing to peek." << endl;
            return;
        }
        cout << "Top element is: " << top->data << endl;
    }
    void traverse() const {
        if (isEmpty()) {
            cout << "Stack is empty." << endl;
            return;
        }
        Node* temp = top;
        cout << "Stack elements are: ";
        while (temp != nullptr) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
}

```

```
bool isEmpty() const {  
    return top == nullptr;  
}  
};  
int main() {  
    Stack stack;  
  
    stack.push(10);  
    stack.push(20);  
    stack.push(30);  
  
    stack.traverse();  
    stack.peek();  
  
    stack.pop();  
    stack.pop();  
    stack.pop();  
    stack.pop();  
  
    stack.traverse();  
    return 0;  
}  
Output –
```

```
10 pushed onto the stack.  
20 pushed onto the stack.  
30 pushed onto the stack.  
Stack elements are: 30 20 10  
Top element is: 30  
30 popped from the stack.  
20 popped from the stack.  
10 popped from the stack.  
Stack is empty, nothing to pop.  
Stack is empty.
```

```
=== Code Execution Successful ===
```

PROGRAM - 7

AIM – Write a program to perform Binary search in the array

S/W used – Online C++ compiler

Algorithm –

Source code –

```
#include <iostream>
using namespace std;
```

```
int binarySearch(int arr[], int size, int target) {
    int left = 0;
    int right = size - 1;
```

```
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid;
        }
        if (arr[mid] < target) {
            left = mid + 1;
        }
    }
}
```

```
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

int main() {
    int arr[] = {2, 3, 4, 10, 40};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 10;

    int result = binarySearch(arr, size, target);

    if (result != -1) {
        cout << "Element found at index " << result << endl;
    } else {
        cout << "Element not found" << endl;
    }
    return 0;
}
```

Output -

```
Element found at index 3
```

```
=== Code Execution Successful ===
```

PROGRAM - 8

AIM – Write a program to Create two arrays and merge them into third one in sorted manner
Algorithm –

Source code –

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> array1 = {1, 3, 5, 7, 9};
    std::vector<int> array2 = {2, 4, 6, 8, 10};

    std::vector<int> mergedArray;
    mergedArray.reserve(array1.size() + array2.size());

    mergedArray.insert(mergedArray.end(), array1.begin(), array1.end());
    mergedArray.insert(mergedArray.end(), array2.begin(), array2.end());

    std::sort(mergedArray.begin(), mergedArray.end());

    std::cout << "Merged and sorted array: ";
    for (const int& num : mergedArray) {
        std::cout << num << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Output –

```
Merged and sorted array: 1 2 3 4 5 6 7 8 9 10

=== Code Execution Successful ===
```

PROGRAM - 9

AIM – Write a program to Find all the subarrays having sum zero in C++

Algorithm –

Source code –

```
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

void findSubarraysWithZeroSum(const vector<int>& arr) {
    unordered_map<int, vector<int>> sumMap;
    int sum = 0;

    for (int i = 0; i < arr.size(); i++) {
        sum += arr[i];

        if (sum == 0) {
            cout << "Subarray with zero sum: ";
            for (int j = 0; j <= i; j++) {
                cout << arr[j] << " ";
            }
        }
    }
}
```

```

        cout << endl;
    }
    if (sumMap.find(sum) != sumMap.end()) {
        for (int startIndex : sumMap[sum]) {
            cout << "Subarray with zero sum: ";
            for (int k = startIndex + 1; k <= i; k++) {
                cout << arr[k] << " ";
            }
            cout << endl;
        }
    }
    sumMap[sum].push_back(i);
}
}
int main() {
    vector<int> arr = {4, 2, -3, 1, 6, -1, 2, -2, 2};

    cout << "Finding all subarrays with zero sum:" << endl;
    findSubarraysWithZeroSum(arr);
    return 0;
}

```

Output –

```

Finding all subarrays with zero sum:
Subarray with zero sum: 2 -3 1
Subarray with zero sum: 2 -2
Subarray with zero sum: -2 2

=== Code Execution Successful ===

```


PROGRAM - 10

AIM – Write a program to Implement stack using Arrays in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

class Stack {
    int top;
    int size;
    int* arr;
public:
    Stack(int s) {
        size = s;
        top = -1;
        arr = new int[size];
    }
    void push(int value) {
        if (top == size - 1) {
            cout << "Stack is full\n";
        } else {
            arr[++top] = value;
            cout << "Pushed: " << value << endl;
        }
    }
    void pop() {
```

```

        if (top == -1) {
            cout << "Stack is empty\n";
        } else {
            cout << "Popped: " << arr[top--] << endl;
        }
    }
    void peek() {
        if (top == -1) {
            cout << "Stack is empty\n";
        } else {
            cout << "Top element: " << arr[top] << endl;
        }
    }
    void display() {
        if (top == -1) {
            cout << "Stack is empty\n";
        } else {
            for (int i = 0; i <= top; i++) {
                cout << arr[i] << " ";
            }
            cout << endl;
        }
    }
};

int main() {
    Stack s(5);

    s.push(10);
    s.push(20);
    s.push(30);

    s.display();
    s.pop();
    s.peek();

    s.display();
    return 0;
}

```

Output –

```

Pushed: 10
Pushed: 20
Pushed: 30
10 20 30
Popped: 30
Top element: 20
10 20

```

PROGRAM - 11

AIM – Write a program to Implement queue using Arrays in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

class Queue {
    int front, rear, size;
    int* arr;

public:
    Queue(int s) {
        front = rear = -1;
        size = s;
        arr = new int[size];
    }
    void enqueue(int value) {
        if (rear == size - 1) {
            cout << "Queue is full\n";
        } else {
            if (front == -1) front = 0;
```

```

        arr[++rear] = value;
        cout << "Enqueued: " << value << endl;
    }
}

void dequeue() {
    if (front == -1 || front > rear) {
        cout << "Queue is empty\n";
    } else {
        cout << "Dequeued: " << arr[front++] << endl;
        if (front > rear) front = rear = -1;
    }
}

void display() {
    if (front == -1 || front > rear) {
        cout << "Queue is empty\n";
    } else {
        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
}

};

int main() {
    Queue q(5);

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    q.display();
    q.dequeue();
    q.display();
    return 0;
}

```

Output –

```

Enqueued: 10
Enqueued: 20
Enqueued: 30
10 20 30
Dequeued: 10
20 30

=== Code Execution Successful ===

```

PROGRAM – 12

AIM – Write a program to Create linked list and find the count of duplicate elements in C++
Algorithm –

Source code –

```
#include <iostream>
#include <unordered_map>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void insert(Node** head, int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
```

```

        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

int countDuplicates(Node* head) {
    unordered_map<int, int> freqMap;
    Node* temp = head;
    int duplicateCount = 0;

    while (temp != NULL) {
        freqMap[temp->data]++;
        temp = temp->next;
    }
    for (auto pair : freqMap) {
        if (pair.second > 1) {
            duplicateCount++;
        }
    }
    return duplicateCount;
}

int main() {
    Node* head = NULL;
    int n, value;

    cout << "Enter the number of elements in the linked list: ";
    cin >> n;
    cout << "Enter the elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> value;
        insert(&head, value);
    }
    cout << "Number of duplicate elements: " << countDuplicates(head) << endl;
    return 0;
}

```

Output –

```

Enter the number of elements in the linked list: 6
Enter the elements:
22 33 44 55 22 33 34
Number of duplicate elements: 2

=== Code Execution Successful ===

```

PROGRAM - 13

AIM – Write a program to Create a linear queue using LL and implement insertion, deletion and display in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class Queue {
    Node* front;
    Node* rear;

public:
    Queue() : front(nullptr), rear(nullptr) {}

    void enqueue(int value) {
        Node* newNode = new Node{value, nullptr};
        if (rear) {
            rear->next = newNode;
        } else {
            front = newNode;
        }
        rear = newNode;
    }
};
```

```

        cout << "Enqueued: " << value << endl;
    }
    void dequeue() {
        if (!front) {
            cout << "Queue is empty\n";
            return;
        }
        Node* temp = front;
        front = front->next;
        if (!front) {
            rear = nullptr;
        }
        cout << "Dequeued: " << temp->data << endl;
        delete temp;
    }
    void display() {
        Node* temp = front;
        if (!temp) {
            cout << "Queue is empty\n";
            return;
        }
        while (temp) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};

int main() {
    Queue q;
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    q.display();
    q.dequeue();
    q.display();
    return 0;
}

```

Output –

```

Enqueued: 10
Enqueued: 20
Enqueued: 30
10 20 30
Dequeued: 10
20 30

```


PROGRAM - 14

AIM – Write a program to Implement Sparse Matrix using LL in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

struct Node {
    int row, col, value;
    Node* next;
};

Node* createNode(int row, int col, int value) {
    Node* newNode = new Node{row, col, value, nullptr};
    return newNode;
}

void insert(Node** head, int row, int col, int value) {
    if (value != 0) {
        Node* newNode = createNode(row, col, value);
        newNode->next = *head;
        *head = newNode;
    }
}

void display(Node* head, int numRows, int numCols) {
    Node* temp;
    for (int i = 0; i < numRows; i++) {
```

```

        for (int j = 0; j < numCols; j++) {
            temp = head;
            bool found = false;
            while (temp) {
                if (temp->row == i && temp->col == j) {
                    cout << temp->value << " ";
                    found = true;
                    break;
                }
                temp = temp->next;
            }
            if (!found) cout << "0 ";
        }
        cout << endl;
    }
}

int main() {
    Node* head = nullptr;
    int numRows = 4, numCols = 5;
    insert(&head, 0, 1, 5);
    insert(&head, 1, 0, 8);
    insert(&head, 2, 3, 10);
    insert(&head, 3, 4, 15);

    display(head, numRows, numCols);
    return 0;
}

```

Output -

```

0 5 0 0 0
8 0 0 0 0
0 0 0 10 0
0 0 0 0 15

=== Code Execution Successful ===

```

PROGRAM -

AIM – Write a program to Implement Binary tree and Traverse (pre,in,postorder) using recursion in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

void preorder(Node* root) {
    if (root == nullptr) {
        return;
    }
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

```

}
void inorder(Node* root) {
    if (root == nullptr) {
        return;
    }
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
void postorder(Node* root) {
    if (root == nullptr) {
        return;
    }
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->left = new Node(6);
    root->right->right = new Node(7);
    cout << "Preorder traversal: ";
    preorder(root);
    cout << "\n";
    cout << "Inorder traversal: ";
    inorder(root);
    cout << "\n";
    cout << "Postorder traversal: ";
    postorder(root);
    cout << "\n";
    return 0;
}

```

Output -

```

Preorder traversal: 1 2 4 5 3 6 7
Inorder traversal: 4 2 5 1 6 3 7
Postorder traversal: 4 5 2 6 7 3 1

```

```

=== Code Execution Successful ===

```

PROGRAM -

AIM – Write a program to Implement Binary search tree wuth information about automobile and perform insertion, deletion and traversal in C++

Algorithm –

Source code –

```
#include <iostream>
#include <string>
using namespace std;

struct Automobile {
    string make, model;
    int year;
    Automobile(string m, string mo, int y) : make(m), model(mo), year(y) {}
};

struct Node {
    Automobile* autoInfo;
    Node* left, * right;
    Node(Automobile* info) : autoInfo(info), left(nullptr), right(nullptr) {}
};
```

```

class BST {
private:
    Node* root;

    void insert(Node*& node, Automobile* autoInfo) {
        if (!node) node = new Node(autoInfo);
        else if (autoInfo->year < node->autoInfo->year) insert(node->left, autoInfo);
        else insert(node->right, autoInfo);
    }
    Node* deleteNode(Node* node, int year) {
        if (!node) return node;
        if (year < node->autoInfo->year) node->left = deleteNode(node->left, year);
        else if (year > node->autoInfo->year) node->right = deleteNode(node->right,
year);
        else {
            if (!node->left) {
                Node* temp = node->right;
                delete node;
                return temp;
            }
            if (!node->right) {
                Node* temp = node->left;
                delete node;
                return temp;
            }
            Node* temp = minValueNode(node->right);
            node->autoInfo = temp->autoInfo;
            node->right = deleteNode(node->right, temp->autoInfo->year);
        }
        return node;
    }
    Node* minValueNode(Node* node) {
        while (node && node->left) node = node->left;
        return node;
    }
    void inorder(Node* node) {
        if (node) {
            inorder(node->left);
            cout << "Make: " << node->autoInfo->make
                << ", Model: " << node->autoInfo->model
                << ", Year: " << node->autoInfo->year << endl;
        }
    }
}

```

```

        inorder(node->right);
    }
}
public:
    BST() : root(nullptr) {}
    void insert(string make, string model, int year) {
        Automobile* autoInfo = new Automobile(make, model, year);
        insert(root, autoInfo);
    }
    void deleteNode(int year) {
        root = deleteNode(root, year);
    }
    void inorder() {
        inorder(root);
    }
};

int main() {
    BST tree;
    tree.insert("Toyota", "Camry", 2020);
    tree.insert("Honda", "Civic", 2018);
    tree.insert("Ford", "Mustang", 2021);
    tree.insert("Chevrolet", "Malibu", 2019);
    tree.insert("Tesla", "Model S", 2022);

    cout << "Inorder traversal of the BST:" << endl;
    tree.inorder();
    tree.deleteNode(2018);
    cout << "\nInorder traversal after deletion:" << endl;
    tree.inorder();
    return 0;
}

```

Output -

```

Inorder traversal of the BST:
Make: Honda, Model: Civic, Year: 2018
Make: Chevrolet, Model: Malibu, Year: 2019
Make: Toyota, Model: Camry, Year: 2020
Make: Ford, Model: Mustang, Year: 2021
Make: Tesla, Model: Model S, Year: 2022

Inorder traversal after deletion:
Make: Chevrolet, Model: Malibu, Year: 2019
Make: Toyota, Model: Camry, Year: 2020
Make: Ford, Model: Mustang, Year: 2021
Make: Tesla, Model: Model S, Year: 2022

```

PROGRAM -

AIM – Write a program to Remove a loop from LL in C++

Algorithm –

Source code –

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

void removeLoop(Node* loopNode) {
    Node *ptr1 = loopNode, *ptr2 = loopNode;
    int k = 1;
    while (ptr1->next != ptr2) {
        ptr1 = ptr1->next;
        k++;
    }
    ptr1 = loopNode;
    for (int i = 0; i < k; i++) ptr1 = ptr1->next;
    while (ptr1 != ptr2) {
        ptr1 = ptr1->next;
        ptr2 = ptr2->next;
    }
}
```



```

        while (ptr1->next != ptr2) ptr1 = ptr1->next;
        ptr1->next = nullptr;
    }
    void detectAndRemoveLoop(Node* head) {
        Node *slow = head, *fast = head;
        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
            if (slow == fast) {
                removeLoop(slow);
                return;
            }
        }
    }
    void printList(Node* head) {
        while (head) {
            std::cout << head->data << " -> ";
            head = head->next;
        }
        std::cout << "nullptr\n";
    }
    int main() {
        Node* head = new Node(1);
        head->next = new Node(2);
        head->next->next = new Node(3);
        head->next->next->next = new Node(4);
        head->next->next->next->next = new Node(5);
        head->next->next->next->next->next = head->next;

        detectAndRemoveLoop(head);
        printList(head);

        return 0;
    }

```

Output -

```
1 -> 2 -> 3 -> 4 -> nullptr
```

```
=== Code Execution Successful ===
```

PROGRAM -

AIM – Write a program to Segregate even and odd node in LL in C++

Algorithm –

Source code –

```
#include <iostream>
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};
Node* segregateEvenOdd(Node* head) {
    if (!head) return nullptr;

    Node *evenHead = nullptr, *oddHead = nullptr, *evenTail = nullptr, *oddTail =
    nullptr;

    for (Node* current = head; current; current = current->next) {
        if (current->data % 2 == 0) {
            if (!evenHead) evenHead = evenTail = current;
            else evenTail = evenTail->next = current;
        } else {
            if (!oddHead) oddHead = oddTail = current;
```

```

        else oddTail = oddTail->next = current;
    }
}
if (evenTail) evenTail->next = oddHead;
if (oddTail) oddTail->next = nullptr;

return evenHead ? evenHead : oddHead;
}

void printList(Node* head) {
    for (Node* current = head; current; current = current->next)
        std::cout << current->data << " ";
    std::cout << std::endl;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    std::cout << "Original list: ";
    printList(head);

    head = segregateEvenOdd(head);

    std::cout << "Segregated list: ";
    printList(head);
    return 0;
}

```

Output -

```

Original list: 1 2 3 4 5
Segregated list: 2 4 1 3 5

=== Code Execution Successful ===

```

PROGRAM -

AIM – Write a program to Find height of a binary tree in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

int height(TreeNode* root) {
    if (root == nullptr) {
        return -1;
    }
    return max(height(root->left), height(root->right)) + 1;
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
```

```
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(5);

cout << "Height of the binary tree: " << height(root) << endl;
return 0;
}
```

Output -

```
Height of the binary tree: 2

=== Code Execution Successful ===
```

PROGRAM -

AIM – Write a program to Generate min_heap and max_heap in C++

Algorithm –

Source code –

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

struct MinHeapComparator {
    bool operator()(int a, int b) {
        return a > b;
    }
};

int main() {
    // Min-Heap
    priority_queue<int, vector<int>, MinHeapComparator> minHeap;
    minHeap.push(5);
    minHeap.push(3);
    minHeap.push(8);
    minHeap.push(1);
    minHeap.push(6);
```

```

    cout << "Min-Heap elements in sorted order: ";
    while (!minHeap.empty()) {
        cout << minHeap.top() << " ";
        minHeap.pop();
    }
    cout << endl;

    // Max-Heap
    priority_queue<int> maxHeap;
    maxHeap.push(5);
    maxHeap.push(3);
    maxHeap.push(8);
    maxHeap.push(1);
    maxHeap.push(6);

    cout << "Max-Heap elements in sorted order: ";
    while (!maxHeap.empty()) {
        cout << maxHeap.top() << " ";
        maxHeap.pop();
    }
    cout << endl;
    return 0;
}

```

Output -

```

Min-Heap elements in sorted order: 1 3 5 6 8
Max-Heap elements in sorted order: 8 6 5 3 1

=== Code Execution Successful ===

```

PROGRAM -

AIM –Implement Radix sort in C++

Algorithm –

Source code –

```
#include <iostream>

void countingSort(int arr[], int size, int exp) {
    int output[size];
    int count[10] = {0};

    for (int i = 0; i < size; i++) {
        count[(arr[i] / exp) % 10]++;
    }

    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }

    for (int i = size - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }
}
```



```

        for (int i = 0; i < size; i++) {
            arr[i] = output[i];
        }
    }

void radixSort(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    for (int exp = 1; max / exp > 0; exp *= 10) {
        countingSort(arr, size, exp);
    }
}

int main() {
    int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
    int size = sizeof(arr) / sizeof(arr[0]);

    radixSort(arr, size);

    std::cout << "Sorted array: \n";
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}

```

Output -

```

Sorted array:
2 24 45 66 75 90 170 802

=== Code Execution Successful ===

```

PROGRAM -

AIM – Implement selection sort, bubble sort, insertion sort, merge sort, quick sort and heap sort using array as a data structure in C++

Algorithm –

Source code –

```
#include <iostream>
#include <algorithm>
using namespace std;
```

```

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) if (arr[j] < arr[minIndex]) minIndex = j;
        swap(arr[i], arr[minIndex]);
    }
}

```

// 2. Bubble Sort

```

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) for (int j = 0; j < n - i - 1; j++) if (arr[j] > arr[j + 1])
swap(arr[j], arr[j + 1]);
}

```

// 3. Insertion Sort

```

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) arr[j + 1] = arr[j], j--;
        arr[j + 1] = key;
    }
}

```

// 4. Merge Sort

```

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid); mergeSort(arr, mid + 1, right);
        int n1 = mid - left + 1, n2 = right - mid;
        int leftArr[n1], rightArr[n2];
        for (int i = 0; i < n1; i++) leftArr[i] = arr[left + i];
        for (int i = 0; i < n2; i++) rightArr[i] = arr[mid + 1 + i];
        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) arr[k++] = (leftArr[i] <= rightArr[j]) ? leftArr[i++] :
rightArr[j++];
        while (i < n1) arr[k++] = leftArr[i++];
        while (j < n2) arr[k++] = rightArr[j++];
    }
}

```

// 5. Quick Sort

```

int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++) if (arr[j] <= pivot) swap(arr[++i], arr[j]);
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// 6. Heap Sort
void heapify(int arr[], int n, int i) {
    int largest = i, left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) swap(arr[i], arr[largest]), heapify(arr, n, largest);
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) swap(arr[0], arr[i]), heapify(arr, i, 0);
}

// Helper function to print the array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    int arrCopy[n];

    // Selection Sort
    copy(begin(arr), end(arr), arrCopy);
    selectionSort(arrCopy, n); cout << "Selection Sort: "; printArray(arrCopy, n);
}

```

```

// Bubble Sort
copy(begin(arr), end(arr), arrCopy);
bubbleSort(arrCopy, n); cout << "Bubble Sort: "; printArray(arrCopy, n);

// Insertion Sort
copy(begin(arr), end(arr), arrCopy);
insertionSort(arrCopy, n); cout << "Insertion Sort: "; printArray(arrCopy, n);

// Merge Sort
copy(begin(arr), end(arr), arrCopy);
mergeSort(arrCopy, 0, n - 1); cout << "Merge Sort: "; printArray(arrCopy, n);

// Quick Sort
copy(begin(arr), end(arr), arrCopy);
quickSort(arrCopy, 0, n - 1); cout << "Quick Sort: "; printArray(arrCopy, n);

// Heap Sort
copy(begin(arr), end(arr), arrCopy);
heapSort(arrCopy, n); cout << "Heap Sort: "; printArray(arrCopy, n);

return 0;
}

```

Output -

```

Selection Sort: 11 12 22 25 34 64 90
Bubble Sort: 11 12 22 25 34 64 90
Insertion Sort: 11 12 22 25 34 64 90
Merge Sort: 11 12 22 25 34 64 90
Quick Sort: 11 12 22 25 34 64 90
Heap Sort: 11 12 22 25 34 64 90

=== Code Execution Successful ===

```

PROGRAM -

AIM – Implement searching using hashing method in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

class HashTable {
    int *table;
    int size;

public:
    HashTable(int s) {
        size = s;
        table = new int[size];
        for (int i = 0; i < size; i++) {
            table[i] = -1; // Initialize all slots to -1 (indicating empty)
        }
    }
};
```

```

    }
}

int hash(int key) {
    return key % size;
}

void insert(int key) {
    int index = hash(key);

    while (table[index] != -1) {
        index = (index + 1) % size; // Move to the next index
    }

    table[index] = key;
    cout << "Inserted " << key << " at index " << index << endl;
}

bool search(int key) {
    int index = hash(key);

    while (table[index] != -1) {
        if (table[index] == key) {
            cout << "Found " << key << " at index " << index << endl;
            return true;
        }
        index = (index + 1) % size; // Move to the next index
    }

    cout << key << " not found in the table" << endl;
    return false;
}

~HashTable() {
    delete[] table;
}

};

int main() {
    HashTable ht(10); // Create a hash table with 10 slots

    ht.insert(15); // Insert 15

```

```
        ht.insert(25); // Insert 25
        ht.insert(35); // Insert 35
        ht.insert(45); // Insert 45

        ht.search(25); // Search for 25
        ht.search(50); // Search for 50

        return 0;
    }
}
```

Output -

```
Inserted 15 at index 5
Inserted 25 at index 6
Inserted 35 at index 7
Inserted 45 at index 8
Found 25 at index 6
50 not found in the table

=== Code Execution Successful ===
```


PROGRAM -

AIM – Create a graph and perform DFS and BFS traversals in C++

Algorithm –

Source code –

```
#include <iostream>
using namespace std;

#define MAX_VERTICES 10

class Graph {
private:
    bool adjMatrix[MAX_VERTICES][MAX_VERTICES];
    bool visited[MAX_VERTICES];
    int numVertices;

public:
    Graph(int vertices) {
        numVertices = vertices;
        for (int i = 0; i < vertices; i++) {
            visited[i] = false;
            fill(adjMatrix[i], adjMatrix[i] + vertices, false);
        }
    }
};
```

```

    }

    void addEdge(int u, int v) {
        adjMatrix[u][v] = adjMatrix[v][u] = true;
    }

    void DFS(int v) {
        visited[v] = true;
        cout << v << " ";
        for (int i = 0; i < numVertices; i++) {
            if (adjMatrix[v][i] && !visited[i]) DFS(i);
        }
    }

    void BFS(int start) {
        int queue[MAX_VERTICES], front = 0, rear = -1;
        visited[start] = true;
        queue[++rear] = start;
        while (front <= rear) {
            int v = queue[front++];
            cout << v << " ";
            for (int i = 0; i < numVertices; i++) {
                if (adjMatrix[v][i] && !visited[i]) {
                    visited[i] = true;
                    queue[++rear] = i;
                }
            }
        }
    }

    void resetVisited() {
        fill(visited, visited + numVertices, false);
    }
};

int main() {
    Graph g(6);
    g.addEdge(0, 1); g.addEdge(0, 2); g.addEdge(1, 3); g.addEdge(1, 4); g.addEdge(2,
5);

    cout << "DFS from 0: "; g.DFS(0); cout << endl;

```

```
g.resetVisited();  
cout << "BFS from 0: "; g.BFS(0); cout << endl;  
  
return 0;  
}
```

Output -

```
DFS from 0: 0 1 3 4 2 5  
BFS from 0: 0 1 2 3 4 5  
  
=== Code Execution Successful ===
```